

Zapier at the center of your pipelines

The Green-IT Choice: Zapier vs. WPipe 🌱⚡

The context

Every Zap costs energy and money. Cloud-heavy automation is the "Gas Guzzler" of the digital age. WPipe is the Electric Vehicle of Pipelines.

Why engineers shift to wpipe

- Ultra-Low Memory: < 50MB RAM.
- Local First: Run it on-prem or in tiny containers. No heavy cloud overhead.
- High Performance: Parallel execution that doesn't burn your CPU (or your wallet).

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="sync", retry_count=3, retry_delay=5)
def sync(data):
    return {"status": push_to_crm(data["lead"])}

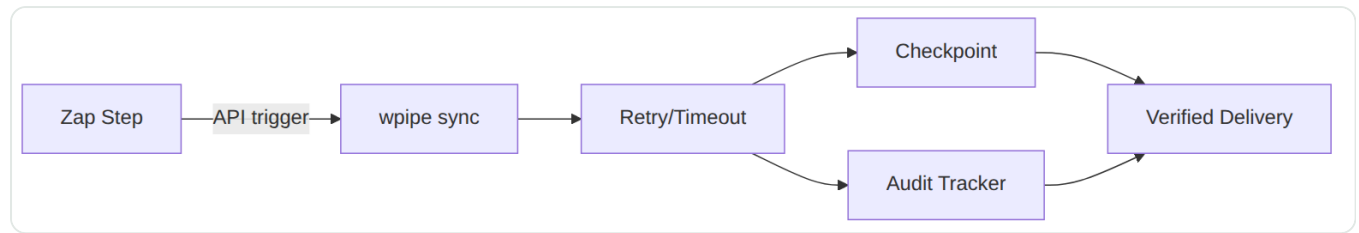
pipe = Pipeline(pipeline_name="zap_to_wpipe", tracking_db="sync.db")
pipe.set_steps([sync])
```

At a glance

Aspect	Legacy / Zapier	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Zapier** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.